

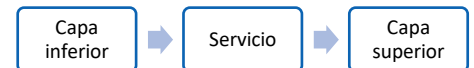
TEMA 1: INTRODUCCIÓN AL DESARROLLO DE APLICACIONES EMPRESARIALES

INTRODUCCIÓN Y CARACTERÍSTICAS DE LAS APLICACIONES EMPRESARIALES

- Aplicación empresarial → tipo de aplicación que aborda necesidades críticas (no forzosamente en empresas)
- Deben satisfacer una serie de requisitos
 - Escalabilidad
 - Capacidad de soportar + usuarios o + carga al aumentar los recursos disponibles (ej. hardware)
 - Alta disponibilidad
 - Tolerancia a fallos mediante replicación de recursos
 - Transaccionalidad
 - Propiedades ACID
 - Ej. transferencia bancaria
 - Seguridad
 - Que no todos los usuarios puedan acceder a la misma funcionalidad
- Funcionalidad de las aplicaciones modernas
 - Dependen de otras aplicaciones (accesibles a través de la red) para proporcionar sus servicios a los usuarios → no funcionan de forma aislada
 - Ofrecen servicios a otras aplicaciones
 - Poco control sobre otras aplicaciones
 - No es posible modificarlas para que se ajusten a las necesidades de tu aplicación
 - Pueden ser de otras empresas (ej. B2B, cloud, ...)
- Problemas
 - Acceso a través de la red
 - Integración con aplicaciones desarrolladas con otras tecnologías
 - Evolución independiente de las distintas aplicaciones → minimizar el impacto de cambios

DISEÑO POR CAPAS

- Técnica de diseño + usada en Ciencias de la Computación (ej. arquitecturas de protocolos de redes)
- Una capa “inferior” proporciona servicio a otra capa “superior”
- El servicio se define mediante un “contrato de servicio” (similar a interfaz JAVA)
- Permite independizar el software de ambas capas
 - A la capa superior no le importara cómo se implementa el servicio
 - Los cambios en la manera de implementar una capa no afectan a la otra
- Facilita el mantenimiento, escalabilidad y tolerancia a fallos
- Ventajas
 - No es necesario conocer las tecnologías de otras capas
 - Capas desarrolladas en paralelo
 - Facilita el mantenimiento del software
 - Cambios en la implementación de una capa no afecta a los cambios en el resto de las capas (ej. uso de nuevas tecnologías)
 - Facilita la escalabilidad y tolerancia a fallos → aumento de recursos sólo en las capas necesarias
- Riesgos
 - Software + complejo
 - Optimización de la función de las capas si conocen cómo funciona otra capa (esto solo ocurre a veces)



ARQUITECTURA TÍPICA

- Separación entre las capas “modelo” e “interfaz”
- Capa Modelo
 - Conjunto de clases que implementan la **lógica** de negocio de los **casos de uso** de la app
 - Subcapas
 - Capa Acceso a Datos
 - Bases de datos
 - Otras aplicaciones
 - Capa Lógica de Negocio
 - Implementa la lógica de negocio
 - Usa la capa de acceso a datos para leer/escribir los datos necesarios
 - Independiente de la interfaz
- 2 casos
 - Usuario “humano”
 - Capa Interfaz Gráfica
 - Permite a los usuarios utilizar la funcionalidad de la capa modelo
 - Aplicaciones usuarias
 - Capa Servicios
 - Interfaz programática orientada a que otras aplicaciones remotas utilicen la funcionalidad de la capa modelo
 - Las aplicaciones remotas invocan la capa Servicios a través de la capa Acceso a Servicios

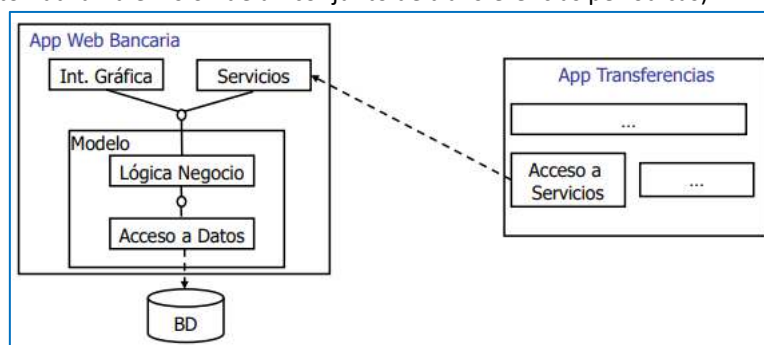
EJEMPLO: APLICACIÓN WEB BANCARIA

- Capa Modelo → conjunto de clases que implementan los casos de uso (ej. crear cuentas, destruirlas, encontrarlas por distintos criterios, ordenar transferencias, etc.)
 - Capa Acceso a Datos → utiliza una BD para guardar la información de cuentas y clientes
 - Capa Lógica de Negocio → implementa la lógica de los casos de uso sin preocuparse de cómo se almacenan los datos
 - Ej. caso de uso transferencia

```

Leer saldoOrigen // Llamada a capa acceso a datos
Si saldoOrigen > importe {
    nuevoSaldoOrigen = saldoOrigen - importe
    // Llamada a capa acceso a datos
    Escribir nuevoSaldoOrigen
    // Llamada a capa acceso a datos
    Escribir nuevoSaldoDestino a su valor actual + importe
} si no lanzar error saldo insuficiente
  
```

- Capa Interfaz Gráfica → interfaz web que utiliza la capa modelo para permitir a los usuarios crear cuentas, buscarlas, hacer transferencias, etc.
- Capa Servicios → ofrece una API que puede ser invocada remotamente por otras aplicaciones (ej. una aplicación que necesita automatizar la emisión de un conjunto de transferencias periódicas)



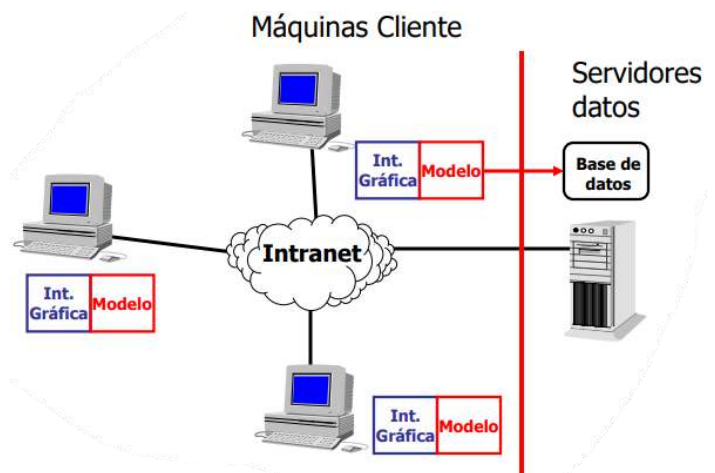
VENTAJAS POTENCIALES DE LA SEPARACIÓN

- Capas desarrolladas por personas con un perfil distinto
- Posible reusar la capa modelo en distintas interfaces gráficas y por distintas aplicaciones
- Cambios en la manera de implementar una capa no afectan a las otras capas
- Cada capa puede ejecutarse en máquinas diferentes (normalmente) → aumenta escalabilidad
- Posible replicar la capa modelo sin necesidad de replicar las aplicaciones consumidoras → facilita escalabilidad y tolerancia a fallos

DISTRIBUCIÓN DE LAS CAPAS

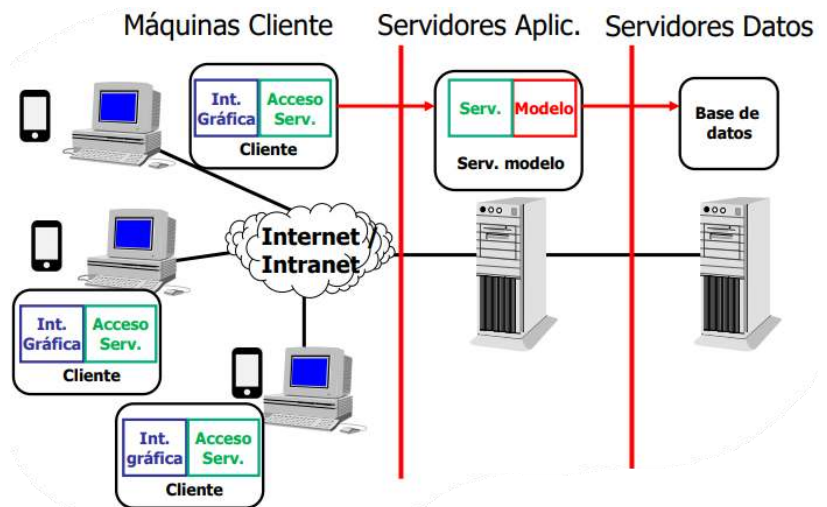
OPCIÓN 1

- No hay tantas capas porque las capas de interfaz gráfica y modelo están en la misma aplicación, por lo que las otras no son necesarias
- Problemas
 - Cambios en la implementación de la capa Modelo → recompilación de toda la aplicación y reinstalación en máquinas cliente
 - Cambios de acceso a fuentes de datos (normalmente BD)
 - Cambios en la lógica del modelo
 - Seguridad → código y accesos a BD desde el cliente
 - Código → se puede recompilar el código y encontrar agujeros para poder acceder a la base de datos
- Solución
 - Modelo en servidor intermedio
 - Cambios en implementación del modelo solo afectan al servidor
 - Clientes de escritorio
 - Sólo disponen de la interfaz gráfica
 - Acceden al servidor que implementa el modelo



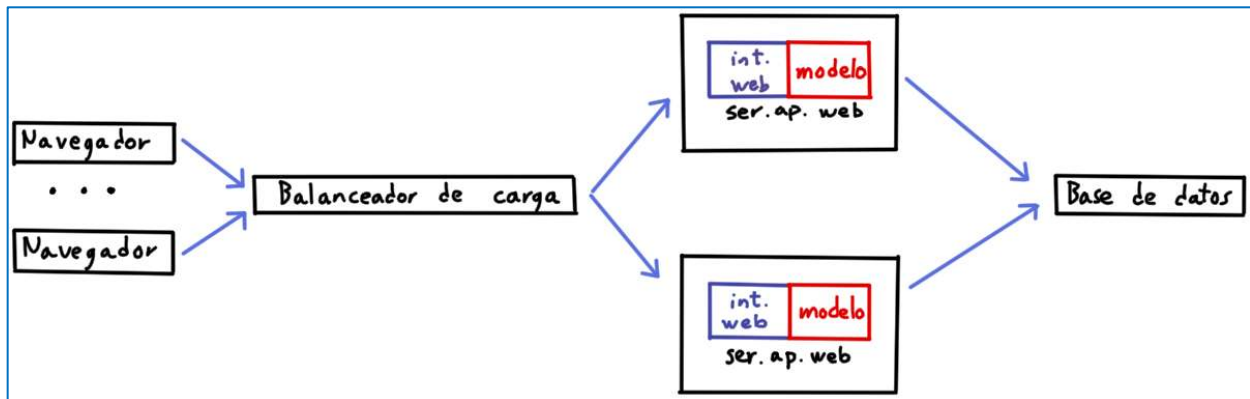
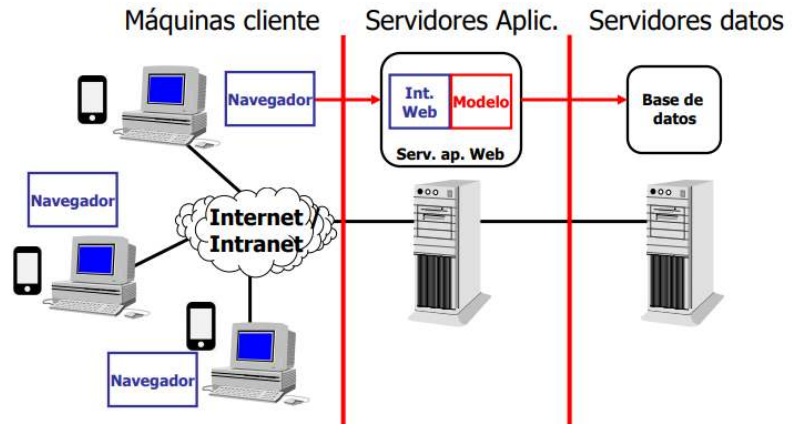
OPCIÓN 2

- Necesaria la Capa Servicios y Capa Acceso a Servicios → hay invocación remota de la capa Modelo
- Problema
 - Cambios en la implementación de la interfaz gráfica → recompilación de la aplicación cliente y reinstalación en máquinas cliente
 - Usar Interfaz Web lo soluciona
- Esta distribución también se le llama "arquitectura en 3 capas" (3 capas hardware no software)
- Ventaja interfaz standalone frente a Interfaz Web
 - Puede proporcionar cierta funcionalidad, incluso con conexión de red intermitente o lenta



OPCIÓN 3

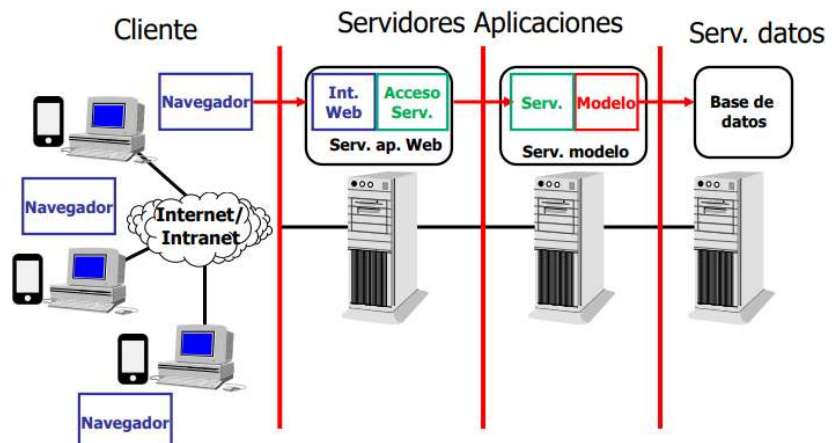
- Las aplicaciones Web del lado servidor se instalan en **servidores de aplicaciones**
- Los cambios en la capa Interfaz Gráfica o en la capa Modelo sólo requieren reinstalar la aplicación en el servidor de aplicaciones
- Los servidores de aplicaciones suelen tener soporte para escalabilidad y disponibilidad (tolerancia a fallos)
 - Se pueden replicar en varias máquinas ("cluster de máquinas")
 - Se emplea un balanceador de carga
 - Recibe peticiones HTTP
 - Envía cada petición HTTP a uno de los servidores de aplicaciones
 - Escalabilidad
 - Para atender + peticiones → hay que añadir + máquinas al cluster
 - Poder aumentar el nº de máquinas replicando el servicio o la aplicación web y poniendo un balanceador de carga



- Disponibilidad
 - Conseguir que las máquinas y los servicios no se caigan, por ejemplo, replicando los servicios (dibujo de arriba)
 - Es tolerante a fallos si por muchas máquinas que se caigan no se cae el servicio (como la udc) (aunque suele haber un límite)
 - Si una máquina se cae todavía quedan otras instancias del servidor de aplicaciones en el cluster
 - Ej. Google

OPCIÓN 4

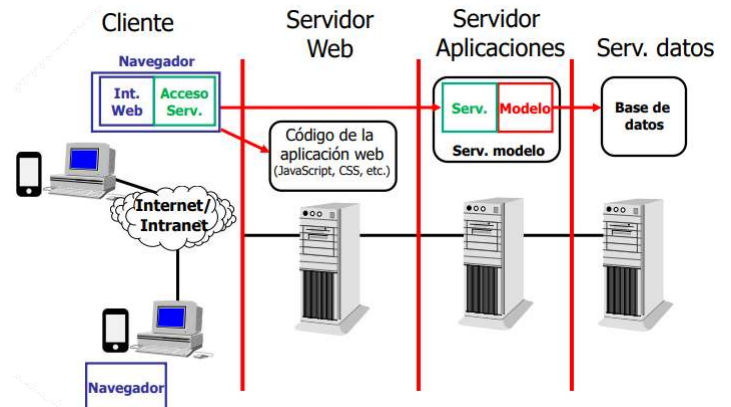
- Distribución conocida como "arquitectura en 4 capas"
- La capa servicios se puede implementar utilizando protocolos y tecnologías usados para el desarrollo de aplicaciones web → necesario ejecutarla dentro de un servidor de aplicaciones
- Ventajas
 - Múltiples aplicaciones (con o sin interfaz gráfica) pueden invocar el modelo sin necesidad de replicar el código del modelo en cada aplicación
 - Permite que la capa Interfaz Gráfica y la capa Modelo están construidas con tecnologías diferentes



- Ejemplo
 - Existe una capa Modelo desarrollada en COBOL
 - La capa modelo es grande, funciona correctamente, lleva muchos años funcionando y manteniéndose
 - Se desea construir una aplicación Web para poder invocar casos de uso de la capa Modelo desde un navegador
 - Se elige una tecnología moderna para implementar la interfaz Web fácilmente (ej. java)
 - No es viable volver a construir la capa Modelo en la misma tecnología que la usada en la interfaz Web (tiempo, coste, razones tecnológicas, etc.)
 - Solución → habilitar la capa Modelo para acceso remoto (ej. Servicios Web)
 - Permite también que otras aplicaciones remotas desarrolladas con tecnologías modernas (aun sin ser interfaz gráfica) utilicen esa funcionalidad

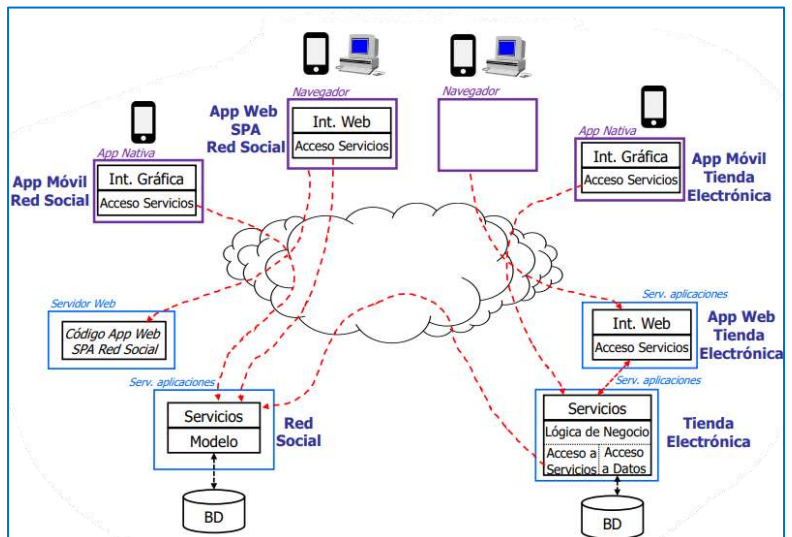
OPCIÓN 5

- Distribución que siguen las aplicaciones web SPA (Single Page Application)
 - El navegador se conecta a un servidor web para obtener todo el código de la aplicación web
 - La aplicación web se ejecuta dentro del navegador de la máquina cliente → necesario utilizar capas Servicios y Acceso a Servicios porque hay invocaciones remotas
- Cambios en la capa Interfaz Gráfica sólo requieren reinstalar la aplicación en el servidor web
- Cambios en la capa Modelo solo requieren reinstalar la aplicación en el servidor de aplicaciones



CASO PRÁCTICO

- La aplicación Tienda Electrónica ofrece los servicios típicos de una tienda de comercio electrónico (ej. buscar productos, añadir al carrito, comprar, ver pedidos, etc.)
 - Dado que se han desarrollado tanto una aplicación Web como una aplicación móvil nativa para interactuar con la tienda, se ha decidido ofrecer la funcionalidad de la capa Modelo a través de una capa Servicios y que ambas aplicaciones accedan al modelo a través de ella
- La aplicación Red Social es una red social desarrollada por otra compañía
 - También ha desarrollado 2 aplicaciones para poder utilizarla: una aplicación móvil nativa y una aplicación web SPA. Al igual que en el caso anterior, ambas aplicaciones acceden a la funcionalidad de la capa Modelo a través de una capa Servicios
 - La capa Servicios también se ha diseñado para que otras aplicaciones puedan interactuar con ella (en la práctica, podría ser otra capa Servicios diferente)



- La tienda electrónica tiene una cuenta en la red social y desea que al ejecutar ciertas operaciones de la capa Modelo, esta interactúa con la Red Social
 - Ejemplos
 - Al añadir un nuevo producto, añadir una publicación a la red social
 - Al obtener los detalles de un producto, obtener también el número de “likes” que tiene la publicación asociada a ese producto en la red social
 - Añadir un comentario cuando queden pocas unidades de un producto, o cuando se reciban nuevas unidades de un producto agotado
 - Etc.
 - El diseño de la capa Modelo incluye
 - Una capa Acceso a Datos para acceder a los datos que guarda en su Base de Datos local (información sobre los productos, usuarios, compras, etc.)
 - Una capa Acceso a Servicios para interactuar con la aplicación Red Social (añadir una publicación, obtener likes de una publicación, añadir un comentario, etc.)
 - Una capa Lógica de Negocio que hace uso de las operaciones ofrecidas por las 2 capas anteriores para implementar la lógica de la aplicación

TECNOLOGÍAS ESTÁNDAR JAVA

- Oracle → **estandariza** (con la ayuda de fabricantes como IBM, Google, etc.) un conjunto de APIs para el desarrollo de aplicaciones Java
 - Existen **múltiples implementaciones** de distintos fabricantes, incluso muchas Open Source
 - Una aplicación construida con estas APIs no depende de una implementación particular
- **Java SE (Java Platform, Standard Edition)**
 - API básica + herramientas básicas (máquina virtual, compilador, etc.)
- **Java ME (Java Platform, Enterprise Edition)**
 - API análoga a Java SE orientada a dispositivos con recursos limitados (ej. televisiones, reproductores Blu-ray, impresoras, etc.)
- **Java EE (Java Platform, Enterprise Edition) / Jakarta EE**
 - Se apoya en Java SE y dispone de APIs para la construcción de aplicaciones empresariales (inclusive aplicaciones Web)
 - Desde 2018 es gestionada por la Eclipse Foundation y pasa a denominarse **Jakarta EE**

IMPLEMENTACIONES DE JAVA/JAKARTA EE

- Aplicaciones desarrolladas usando Java/Jakarta EE → necesitan ser ejecutadas dentro de un servidor de aplicaciones
- Existen varios fabricantes que venden **servidores de aplicaciones certificados Java/Jakarta EE**
 - Ejemplos
 - IBM WebSphere Liberty
 - Red Hat JBoss Enterprise Application Platform
 - Jetty
 - Tomcat
 - Aplicación que sólo usa APIs estándar → es posible instalarla en cualquier servidor de aplicaciones Java/Jakarta EE
 - ¡No se depende de un fabricante!

TECNOLOGÍAS JAVA POR CAPA

TECNOLOGÍAS CAPA ACCESO A DATOS

- API: JDBC (Java DataBase Connectivity)
- Posibilita el acceso a bases de datos relacionales, abstrayendo al programador de las peculiaridades del API de cada base de datos
- El programador puede lanzar consultas (lectura, actualización, inserción y borrado), agrupar consultas en transacciones, etc.
- Existen soluciones de + alto nivel (ej. JPA/Hibernate)

TECNOLOGÍAS CAPA LÓGICA DE NEGOCIO

- No se usa ninguna tecnología de apoyo
- Existen tecnologías que ayudan a implementar funcionalidades útiles en muchas aplicaciones
 - Políticas de transacciones y seguridad
 - Utilidades que simplifican programación (ej. factorías)
- Ej. Spring Framework

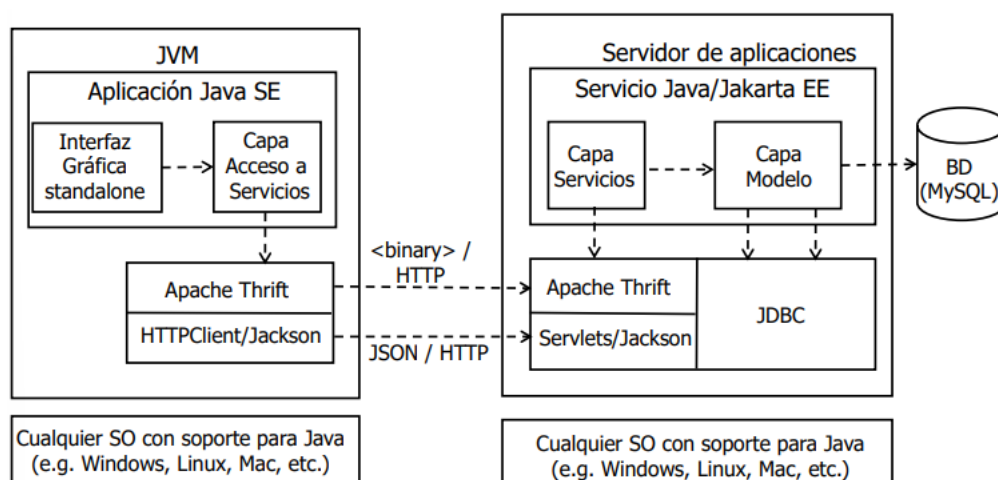
TECNOLOGÍAS CAPA INTERFAZ WEB

- API básica: Servlets
- Existen muchas otras opciones de + alto nivel (ej. JSP)

TECNOLOGÍAS CAPA SERVICIOS

- Servicios web REST
 - Están basados en tecnologías surgidas alrededor de la Web
 - Para enviar datos se usan formatos de representación como XML y JSON
- El Modelo RPC (Remote Procedure Call)
 - Generar automáticamente código que se encarga de abstraer al programador de los detalles de crear y parsear mensajes, y de enviarlos/recibirlos por la red
 - Existen numerosos frameworks que siguen este modelo

ENTORNO DE DESARROLLO DE LA ASIGNATURA



ALTERNATIVAS A LAS TECNOLOGÍAS JAVA

- .NET
 - Define un Common Language Runtime (CLR) y un IL (Intermediate Language) al que todos los lenguajes conformes a .NET compilan → idea similar a la máquina virtual de Java
 - Lenguajes → C#, F#, Visual Basic
 - Tecnologías
 - ADO.NET → para capa modelo
 - ASP.NET → para interfaz web y servicios web
- LAMP → Linux + Apache + MySQL + Perl/PHP/Python
- Ruby on Rails → framework Web para el lenguaje Ruby
- JavaScript
 - Existen multitud de librerías del lado cliente y servidor
 - Node.js → entorno de ejecución JavaScript